Anexo 06 — Entrenamiento PPO del brazo

Archivo: Scripts/rl/train_grab.py

Para que sirvio:
Script de entrenamiento de la policy de agarre. Incluye los callbacks de curriculum: ampliacion progresiva del espacio de aparicion de la bola, activacion de la caida, y rampa de gravedad.

```
"""Entrenamiento PPO para DUMGrabEnv (tarea grab + throw del Pit Droid).

Uso:
    # Smoke (30k steps, fase 1, 4 envs) para validar pipeline
    python Scripts/rl/train_grab.py --phase 1 --steps 30000 --n-envs 4 --name smoke_grab
    # Fase 1 full
    python Scripts/rl/train_grab.py --phase 1 --steps 1000000 --n-envs 16
    # Fase 2 warm-started desde checkpoint de fase 1
    python Scripts/rl/train_grab.py --phase 2 --steps 1000000 --n-envs 16 \\
        --resume runs/grab_phase1_xxx/final.zip
    # Fase 3 warm-started desde fase 2
    python Scripts/rl/train_grab.py --phase 3 --steps 1500000 --n-envs 16 \\
        --resume runs/grab_phase2_xxx/final.zip

Hyperparams por defecto siguen el plan: net=[256,256], n_steps=2048, batch=256,
gamma=0.995, ent_coef=0.005, lr=3e-4 con linear decay a 1e-4. VecNormalize sobre
obs (NO sobre reward).
"""

import argparse
import sys
import time
from pathlib import Path
from datetime import datetime, timedelta

# Para importar rl.envs.grab_env
SCRIPTS_DIR = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(SCRIPTS_DIR))

import torch
torch.set_num_threads(2)

import numpy as np
from stable_baselines3 import PPO
from stable_baselines3.common.vec_env import DummyVecEnv, SubprocVecEnv, VecNormalize
from stable_baselines3.common.callbacks import CheckpointCallback, BaseCallback
from stable_baselines3.common.monitor import Monitor

from rl.envs.grab_env import DUMGrabEnv


class GrabCurriculumV7Callback(BaseCallback):
    """v7 (legacy): activa curriculum diagonal per-episode. Sin uso en v7d."""
    def __init__(self, activate_after_steps: int, verbose: int = 0):
        super().__init__(verbose)
        self.activate_after = int(activate_after_steps)
        self._activated = False

    def _on_step(self) -> bool:
        if self._activated:
            return True
        if self.num_timesteps >= self.activate_after:
            try:
                self.training_env.env_method("set_curriculum_active", True)
            except Exception:
                return True
            self._activated = True
        return True


class GrabCurriculumV7dCallback(BaseCallback):
    """v7d/v7e: ramp lineal per-STEP global de los caps de offset.

    v7e agrega activacion de "falling phase" a fraccion `falling_after_fraction`
    del training (default 0.7 → ultimos 30% con caida ligera g=-0.5).

    log_every controla cada cuanto se imprime el snapshot de progreso
    (default 2M steps por pedido del usuario).
    """
    def __init__(self, total_timesteps: int, max_up: float = 0.17, max_out: float = 0.09,
                 ramp_fraction: float = 0.70, falling_after_fraction: float = 0.70,
                 log_every: int = 2_000_000, verbose: int = 0,
                 gravity_start: float = None, gravity_end: float = None,
                 gravity_ramp_fraction: float = 0.80):
```

```
        super().__init__(verbose)
        self.total = int(total_timesteps)
        self.max_up = float(max_up)
        self.max_out = float(max_out)
        self.ramp_frac = float(ramp_fraction)
        self.falling_after = float(falling_after_fraction)
        self.log_every = int(log_every)
        self._last_logged = 0
        self._falling_activated = False
        # v12: gravity curriculum (opcional). Si None, no ramp.
        self.gravity_start = gravity_start
        self.gravity_end = gravity_end
        self.gravity_ramp_frac = float(gravity_ramp_fraction)

    def _format_progress(self, cur_max_up, cur_max_out, progress):
        # Leer ep_rew_mean del logger interno del modelo si esta disponible
        try:
            ep_rew = self.model.logger.name_to_value.get("rollout/ep_rew_mean", None)
            ep_len = self.model.logger.name_to_value.get("rollout/ep_len_mean", None)
        except Exception:
            ep_rew, ep_len = None, None
        rew_str = f"{ep_rew:+.1f}" if ep_rew is not None else "n/a"
        len_str = f"{ep_len:.1f}" if ep_len is not None else "n/a"
        return (f"steps={self.num_timesteps:,} ({progress:.1%})  "
                f"max_up={cur_max_up*100:.1f}cm  max_out={cur_max_out*100:.1f}cm  "
                f"falling={'YES' if self._falling_activated else 'no '}  "
                f"ep_rew={rew_str}  ep_len={len_str}")

    def _on_step(self) -> bool:
        progress = self.num_timesteps / self.total
        scale = min(progress / self.ramp_frac, 1.0) if self.ramp_frac > 0 else 1.0
        cur_max_up = scale * self.max_up
        cur_max_out = scale * self.max_out
        try:
            self.training_env.env_method("set_curriculum_max_offsets", cur_max_up, cur_max_out)
        except Exception:
            pass

        # v12: gravity ramp si se especifico
        cur_gravity = None
        if self.gravity_start is not None and self.gravity_end is not None:
            g_scale = min(progress / self.gravity_ramp_frac, 1.0) if self.gravity_ramp_frac > 0 else 1.0
            cur_gravity = self.gravity_start + g_scale * (self.gravity_end - self.gravity_start)
            try:
                self.training_env.env_method("set_falling_gravity", cur_gravity)
            except Exception:
                pass

        # Activar falling phase al pasar el umbral (una sola vez)
        if not self._falling_activated and progress >= self.falling_after:
            try:
                self.training_env.env_method("set_falling_active", True)
                self._falling_activated = True
                from datetime import datetime
                stamp = datetime.now().strftime("%H:%M:%S")
                print(f"\n[FALLING PHASE ACTIVADA {stamp}] steps={self.num_timesteps:,}  "
                      f"bola ahora cae con g={'-0.5'} m/s2 (ligera)\n", flush=True)
            except Exception as e:
                if self.verbose:
                    print(f"[v7e] error activando falling: {e}")

        # Log progreso cada log_every steps
        if self.verbose and (self.num_timesteps - self._last_logged) >= self.log_every:
            from datetime import datetime
            stamp = datetime.now().strftime("%H:%M:%S")
            grav_str = f"  g={cur_gravity:.2f}" if cur_gravity is not None else ""
            print(f"[progress {stamp}] {self._format_progress(cur_max_up, cur_max_out, progress)}{grav_str}",
                  flush=True)
            self._last_logged = self.num_timesteps
        return True


class CurriculumF1Callback(BaseCallback):
    """Curriculum in-training de Fase 1: cambia la sub-fase del env segun progreso.
        0-25 %: sub-fase 1a (bola estatica, g=0)
        25-50 %: sub-fase 1b (caida lenta, g=-3)
        50-100 %: sub-fase 1c (caida normal Fase 1, g=-6)
    """
    def __init__(self, total_timesteps: int, verbose: int = 0):
        super().__init__(verbose)
        self.total_timesteps = int(total_timesteps)
        self._last_idx = -1

    def _on_step(self) -> bool:
        # Cada ~10k steps revisamos si cambiar sub-fase (no en cada step para no spamear).
```

```
        if self.num_timesteps % 10000 >= self.training_env.num_envs:
            return True
        progress = self.num_timesteps / self.total_timesteps
        if progress < 0.25:
            idx = 0
        elif progress < 0.50:
            idx = 1
        else:
            idx = 2
        if idx != self._last_idx:
            try:
                self.training_env.env_method("set_subphase", idx)
            except Exception as e:
                if self.verbose:
                    print(f"[curriculum] set_subphase fallo: {e}")
                return True
            self._last_idx = idx
            if self.verbose:
                from datetime import datetime
                stamp = datetime.now().strftime("%H:%M:%S")
                names = ["1a_static (g=0)", "1b_slow (g=-3)", "1c_normal (g=-6)"]
                print(f"[curriculum {stamp}] -> sub-fase {names[idx]} @ steps={self.num_timesteps:,} ({progress:.1%})",
                      flush=True)
        return True

RUNS_DIR = Path(__file__).resolve().parents[2] / "runs"


def make_env(phase=1, seed=0, log_dir=None):
    def _init():
        env = DUMGrabEnv(phase=phase)
        env.reset(seed=seed)
        if log_dir is not None:
            env = Monitor(env, str(log_dir / f"monitor_{seed}.csv"))
        return env
    return _init


def linear_schedule(initial_value, final_value):
    """Schedule lineal de SB3: progress_remaining va 1.0 -> 0.0."""
    def func(progress_remaining):
        return final_value + (initial_value - final_value) * progress_remaining
    return func


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--phase", type=int, default=1, choices=[1, 2, 3],
                   help="Fase del curriculum (1=bola estatica, 2=cae lenta, 3=cae normal+throw)")
    p.add_argument("--steps", type=int, default=1_000_000)
    p.add_argument("--n-envs", type=int, default=16)
    p.add_argument("--resume", type=str, default=None,
                   help="Path al .zip para warm-starting (e.g. policy de fase anterior)")
    p.add_argument("--name", type=str, default=None)
    p.add_argument("--ent-coef", type=float, default=0.005)
    p.add_argument("--learning-rate", type=float, default=3e-4)
    p.add_argument("--lr-final", type=float, default=1e-4,
                   help="LR final del schedule lineal (default 1e-4)")
    p.add_argument("--n-steps", type=int, default=2048)
    p.add_argument("--batch-size", type=int, default=256)
    p.add_argument("--n-epochs", type=int, default=10)
    p.add_argument("--gamma", type=float, default=0.995)
    p.add_argument("--gae-lambda", type=float, default=0.95)
    p.add_argument("--clip-range", type=float, default=0.2)
    p.add_argument("--net-arch", type=str, default="256,256")
    p.add_argument("--no-vecnorm", action="store_true",
                   help="Desactivar VecNormalize (debug)")
    p.add_argument("--curriculum", action="store_true",
                   help="(Fase 1) Activa curriculum 1a->1b->1c segun progreso (25/25/50 %)")
    p.add_argument("--subphase", type=int, default=None, choices=[0, 1, 2],
                   help="(Fase 1) Fijar sub-fase desde el inicio (sin curriculum)")
    p.add_argument("--curriculum-v7-after", type=int, default=None,
                   help="(v7 legacy) Activar el curriculum diagonal del spawn despues de N steps")
    p.add_argument("--curriculum-v7d", action="store_true",
                   help="(v7d) Activar curriculum per-step con ramp lineal y caps asimetricos")
    p.add_argument("--curriculum-v7d-max-up", type=float, default=0.17,
                   help="(v7d) Cap del offset UP (m). Default 0.17m (80% del 21.1cm empirico)")
    p.add_argument("--curriculum-v7d-max-out", type=float, default=0.09,
                   help="(v7d) Cap del offset OUT (m). Default 0.09m (80% del 11.2cm empirico)")
    p.add_argument("--curriculum-v7d-ramp", type=float, default=0.70,
                   help="(v7d) Fraccion del training para alcanzar el max (default 0.7)")
    p.add_argument("--curriculum-v7e-falling-after", type=float, default=1.0,
                   help="(v7e) Fraccion del training tras la cual la bola cae con g lig. Default 1.0 = nunca. 0.7 = ultimos 30%")
```

```
    p.add_argument("--progress-every", type=int, default=2_000_000,
                   help="Cada cuantos steps imprimir progreso del curriculum (default 2M)")
    p.add_argument("--enable-throw", action="store_true",
                   help="(v9) Habilitar el ciclo HELD->THROWN post-grab (en lugar de terminar)")
    p.add_argument("--gravity-start", type=float, default=None,
                   help="(v12) Gravedad de caida al inicio del training (m/s2, negativo)")
    p.add_argument("--gravity-end", type=float, default=None,
                   help="(v12) Gravedad de caida al final (m/s2). Si None, no ramp")
    p.add_argument("--gravity-ramp", type=float, default=0.80,
                   help="(v12) Fraccion del training para alcanzar gravity-end (default 0.80)")
    args = p.parse_args()

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    name = args.name or f"grab_phase{args.phase}_{timestamp}"
    run_dir = RUNS_DIR / name
    run_dir.mkdir(parents=True, exist_ok=True)
    print(f"Run dir: {run_dir}")

    # Crear envs (SubprocVecEnv siempre que n_envs>=2 para paralelismo real)
    if args.n_envs == 1:
        env = DummyVecEnv([make_env(phase=args.phase, seed=0, log_dir=run_dir)])
    else:
        env = SubprocVecEnv([
            make_env(phase=args.phase, seed=i, log_dir=run_dir)
            for i in range(args.n_envs)
        ])

    # Fijar sub-fase manual si se pidio (antes de envolver con VecNormalize)
    if args.subphase is not None and args.phase == 1:
        env.env_method("set_subphase", args.subphase)
        print(f"[train] sub-fase fijada manualmente: {args.subphase}")

    # v9: habilitar throw
    if args.enable_throw:
        env.env_method("set_throw_enabled", True)
        print("[train] throw enabled (HELD->THROWN post-grab)")

    # VecNormalize: solo obs, no reward (magnitudes balanceadas)
    use_vecnorm = not args.no_vecnorm
    if use_vecnorm:
        # Si estamos resumiendo, cargar las stats de VecNormalize del run anterior
        # (mismas dimensiones de obs). Sino, empezar fresh.
        resume_vn = None
        if args.resume:
            candidate = Path(args.resume).parent / "vecnormalize.pkl"
            if candidate.exists():
                resume_vn = candidate
        if resume_vn:
            env = VecNormalize.load(str(resume_vn), env)
            env.training = True
            env.norm_reward = False
            print(f"[train] VecNormalize stats cargadas desde {resume_vn}")
        else:
            env = VecNormalize(env, norm_obs=True, norm_reward=False, clip_obs=10.0)

    # Modelo
    if args.resume:
        print(f"Resuming from {args.resume}")
        # Cargar modelo. Si hay vecnorm stats junto al checkpoint, idealmente cargarlas;
        # las dejamos resetear para simplicidad.
        model = PPO.load(args.resume, env=env)
        # Override hyperparams nuevos (lr schedule)
        model.learning_rate = linear_schedule(args.learning_rate, args.lr_final)
        model._setup_lr_schedule()
        from_scratch = False
    else:
        net_arch = [int(x) for x in args.net_arch.split(",") if x.strip()]
        model = PPO(
            "MlpPolicy",
            env,
            verbose=1,
            tensorboard_log=str(run_dir / "tb"),
            learning_rate=linear_schedule(args.learning_rate, args.lr_final),
            n_steps=args.n_steps,
            batch_size=args.batch_size,
            n_epochs=args.n_epochs,
            gamma=args.gamma,
            gae_lambda=args.gae_lambda,
            clip_range=args.clip_range,
            ent_coef=args.ent_coef,
            vf_coef=0.5,
            policy_kwargs=dict(net_arch=dict(pi=net_arch, vf=net_arch)),
        )
        from_scratch = True

```

```
    checkpoint_cb = CheckpointCallback(
        save_freq=max(args.steps // 10 // args.n_envs, 1),
        save_path=str(run_dir),
        name_prefix="checkpoint",
    )
    callbacks = [checkpoint_cb]
    if args.phase == 1 and args.curriculum:
        callbacks.append(CurriculumF1Callback(total_timesteps=args.steps, verbose=1))
    if args.curriculum_v7_after is not None:
        callbacks.append(GrabCurriculumV7Callback(activate_after_steps=args.curriculum_v7_after, verbose=1))
    if args.curriculum_v7d:
        callbacks.append(GrabCurriculumV7dCallback(
            total_timesteps=args.steps,
            max_up=args.curriculum_v7d_max_up,
            max_out=args.curriculum_v7d_max_out,
            ramp_fraction=args.curriculum_v7d_ramp,
            falling_after_fraction=args.curriculum_v7e_falling_after,
            log_every=args.progress_every,
            verbose=1,
            gravity_start=args.gravity_start,
            gravity_end=args.gravity_end,
            gravity_ramp_fraction=args.gravity_ramp,
        ))

    started_at = datetime.now()
    print("=" * 70)
    print(f"INICIO:        {started_at.strftime('%Y-%m-%d %H:%M:%S')}")
    print(f"Run:           {name}")
    print(f"Modo:          {'FROM SCRATCH' if from_scratch else f'RESUMED from {args.resume}'}")
    print(f"Fase:          {args.phase}")
    print(f"Steps:         {args.steps:,}")
    print(f"N envs:        {args.n_envs}")
    print(f"n_steps/env:   {args.n_steps}  -> rollout total = {args.n_steps * args.n_envs}")
    print(f"batch_size:    {args.batch_size}")
    print(f"n_epochs:      {args.n_epochs}")
    print(f"net_arch:      {args.net_arch}")
    print(f"lr:            {args.learning_rate} -> {args.lr_final} (linear)")
    print(f"gamma:         {args.gamma}")
    print(f"ent_coef:      {args.ent_coef}")
    print(f"VecNormalize:  {'ON (obs only)' if use_vecnorm else 'OFF'}")
    print(f"torch threads: {torch.get_num_threads()}")
    print("=" * 70, flush=True)

    t0 = time.perf_counter()
    model.learn(
        total_timesteps=args.steps,
        callback=callbacks,
        progress_bar=True,
    )
    elapsed = time.perf_counter() - t0

    final_path = run_dir / "final.zip"
    model.save(str(final_path))
    if use_vecnorm:
        env.save(str(run_dir / "vecnormalize.pkl"))
    finished_at = datetime.now()

    print("=" * 70)
    print(f"INICIO:        {started_at.strftime('%Y-%m-%d %H:%M:%S')}")
    print(f"FIN:           {finished_at.strftime('%Y-%m-%d %H:%M:%S')}")
    print(f"DURACION:      {timedelta(seconds=int(elapsed))} ({elapsed:.1f} s)")
    print(f"Modelo final:  {final_path}")
    print("=" * 70)


if __name__ == "__main__":
    main()
```